

README

Part 9: CUDA 内核编程 — 打开深度学习的引擎盖

- > Part 1-8 里 `tensor @ tensor` 一直是黑盒。这一部分我们亲手写 CUDA 内核：
- > 从第一个 vector add, 到手写 matmul 优化阶梯（对比 cuBLAS），再到 Triton 和
- > PyTorch 自定义扩展——最后把它们组装成"毕业内核"：手写 Flash Attention。
- > 参考：[infatoshi/cuda-course](https://github.com/infatoshi/cuda-course)（FreeCodeCamp CUDA Course）

学习目标

完成本部分后，你将能够：

- 画出 GPU 的执行模型（SM / 线程层级 / 存储层次），解释一个内核是怎么跑起来的
- 攀登手写 matmul 的优化阶梯（合并访存 → SMEM tiling → block tiling），用"算力墙 / 内存墙"给每一级定位瓶颈
- 使用 nsys / ncu 做 profiling，用测量数据（而非直觉）决定下一步优化什么
- 编写 Triton 内核（融合算子），说清它与手写 CUDA 的取舍
- 封装自定义算子为 PyTorch 扩展，接入现有训练代码
- 手写 Flash Attention 前向内核（online softmax + causal），对照 SDPA 四后端验收

章节导航

号	章节	内容	对应脚
	[GPU 架构与第一个内核](01_gpu_and_first_kernel.md)	CPU vs GPU、线程层级、够用的 C、CUDA 五步曲	'01' '02'
	[matmul 优化阶梯](02_matmul_optimization.md)	算力墙/内存墙、合并访存、SMEM tiling、block tiling、cuBLAS 对照	'03' '04'
	[Profiling 与 CUDA 库](03_profiling_and_cuda_apis.md)	nsys/ncu、atomics 归约、streams 重叠、cuBLAS（cuDNN 概念带过）	'05' '06'
	[Triton 与 PyTorch 扩展](04_triton_and_extensions.md)	Triton 内核、自定义算子接入 PyTorch、通向 llm.c 的路线图	'07' '08'
	[Flash Attention 毕业内核](05_flash_attention.md)	online softmax 推导、causal 三阶段、SDPA 四后端实测、FlexAttention/SageAttention	'09'

前置知识

必须掌握：

- C 的最低子集：能看懂 `for` 循环和 `a[i]` 数组下标——01 章有"够用的 C"速成节（指针/数组/宏/编译，10 分钟版），零基础也来得及

建议掌握：

- Part 6：Transformer / attention（[Part 6 教程](../Part6_transformer/tutorial/README.md)）——知道"`q @ k^T` 是个算子"这个层面就够了，02 章会分析它为什么是性能大头
- Part 7 03 章 / Part 8：见过 Flash Attention、KV Cache、bf16 autocast 这些词——Part 9 会把它们的"内核原理"补上（02/04 章的连接点）

可选：

- Part 3：诊断工具的思路（先测量、再下结论）——03 章的 GPU profiling 是它的镜像，没学过也不影响
 - 无 GPU 也能学：概念全可读，.cu 脚本可上 Colab/Kaggle 跑（见下方环境表），作业题 1-4 纯 CPU 可完成
- > 本部分与前面所有部分代码完全独立，随时可以直接开始；但学过 Part 7/8 的同学 > 会在"内存墙 → KV Cache / Flash Attention / bf16"这些连接点上获得双倍回报。

学习路线图

Part 1-8 (PyTorch 视角：模型是"用"出来的)

|
| "GPU 到底怎么算 matmul / softmax / attention ?"



Part 9: CUDA 内核（模型是"算"出来的）	
① GPU 架构 + 第一个内核 — 线程层级 / vector add	———→ 01_gpu_and_first_kernel.md
② matmul 优化阶梯 — naive → coalesced → SMEM → tiling	———→ 02_matmul_optimization.md
③ 进阶 CUDA + 库 — atomics / streams / cuBLAS	———→ 03_profiling_and_cuda_apis.md
④ Triton + PyTorch 扩展 — 融合内核 / 自定义算子 / llm.c	———→ 04_triton_and_extensions.md
⑤ Flash Attention — online softmax / causal / SDPA 验收	———→ 05_flash_attention.md

环境要求（与其他部分不同！）

本部分脚本需要 NVIDIA GPU + CUDA Toolkit：

```
`bash
nvidia-smi # ① 驱动 + 显卡
nvcc -V # ② CUDA Toolkit（nvcc 编译器）
python -c "import torch; print(torch.cuda.is_available(), __import__('triton').__version__)"
③ torch(GPU 版) + triton（torch 自带，Linux 下无需单独安装）
```

什么	怎么办
有	`cd courses/Part9_cuda_kernels/scripts && make && make run`，然后跑 07/08/09 三个 .py（09 需独占 GPU，约 2-4 分钟，大头是 autotune 编译
有 GPU	概念照学（教程全可读）；.cu 脚本上 [Colab](https://colab.research.google.com)/Kaggle 免费卡跑；**作业题 1-4 纯 CPU 可完成**
GPU 没 nvcc	装 [CUDA Toolkit](https://developer.nvidia.com/cuda-downloads)（或对应 PyTorch 版本），Colab 也可

多版本 CUDA 共存（本课开发机的真实配置）：

机器上可以同时装多个 CUDA Toolkit（例如 `/usr/local/cuda-11.8` 和 `/usr/local/cuda-12.4`），互不覆盖：

- `PATH` 和 `/usr/local/cuda` 软链接保持指向旧版 → 已有环境（其他项目、脚本）完全不受影响
- 本课的 Makefile 和 PyTorch 扩展脚本会自动选择版本最高的 `/usr/local/cuda-*`
- 想手动指定某个版本：`make NVCC=/usr/local/cuda-11.8/bin/nvcc`，或在 Python 里 `os.environ["CUDA_HOME"] = "/usr/local/cuda-11.8"`
- 安装新版本到独立目录（不动软链接）：

```
`bash
```

```
sudo sh cuda_12.4.1_*.run --silent --toolkit --override --toolkitpath=/usr/local/cuda-12.4
```

装完检查软链接，若被改向新版本，恢复指回旧版：

```
sudo ln -sf /usr/local/cuda-11.8 /usr/local/cuda
```

编译相关坑点（都来自真机实测，撞到时回来看）：

- `unsupported GNU version! gcc versions later than 11...`：CUDA 对宿主 gcc 有版本上限（11.x 最高 gcc-11；12.1+ 原生支持 gcc-12/13）。选中 11.x 时 Makefile/扩展脚本会自动加 `-ccbin gcc-11`，选 12.x 则什么都不用做
- 多版本共存时的运行时库：编译用的 `libcublas` 等也要在运行时被找到，而系统 `ldconfig` 通常只配了旧版路径。Makefile 已把所用版本的 `lib64` 用 `rpath` 焊进二进制（`ldd bin/06_cublas_sgemm` 可验证解析到了哪个版本），无需配 `LD_LIBRARY_PATH`
- PyTorch 扩展的 `CUDA_HOME` 缓存：torch 在 `import` 时就把 `CUDA_HOME` 解析并缓存成模块全局，之后只改 `os.environ` 无效——要同时覆盖 `torch.utils.cpp_extension.CUDA_HOME`（脚本已处理）
- PyTorch 扩展 JIT 编译报 `Ninja is required`：`pip install ninja` 并确保其可执行文件在 `PATH`
- torch 扩展编译产物缓存在 `~/.cache/torch_extensions/`，异常时清掉重编

与原课程的对应：本部分完整覆盖 `cuda-course` 的核心 lecture

（01 生态 / 03 C 复习 / 04 GPU 简介 / 05 first kernels / 06 APIs / 07 faster matmul / 08 Triton / 09 extensions），映射表见

[docs/part9_cuda_kernels_plan.md](../././docs/part9_cuda_kernels_plan.md)。

实测参考（RTX 4090, fp32, 512^3 matmul）

我们的脚本在你机器上跑出来会是类似这样（数字随硬件浮动，看趋势）：

```
L1 uncoalesced 553.6 GFLOPS ← 合并访存做错（对照组）
L2 coalesced 4844.9 GFLOPS ← 只是"读的方式"对了
L3 smem tile 5905.4 GFLOPS ← shared memory 复用
L4 1D blocktile 6967.5 GFLOPS ← 寄存器复用 x8
L5 2D blocktile 8795.2 GFLOPS ← 手写阶梯的尽头
cuBLAS 22163.1 GFLOPS ← Tensor Core + autotune（库的天花板）
```

> △ 小矩阵（ 512^3 全部塞进 L2 cache）会低估 tiling 的收益；教程 02 章解释了为什么
> 大矩阵下差距会拉大。看趋势，别死记数字。

Flash Attention（脚本 09, RTX 4090, torch 2.6.0+cu124 / triton 3.2.0, bf16, B=2 / H=8 / D=64, 前向，预热 10 + 测 50；2026-09-02 共享 GPU 实测）：

T=4096 causal: naive 6.198 ms → 手写 Triton FA 0.279 ms (123 TF, 22.2x)

SDPA 最优 (cudnn) 0.308 ms (112 TF) → 手写版 111%

T=4096 full: naive 3.579 ms → 手写 Triton FA 0.436 ms (158 TF, 8.2x)

SDPA 最优 (cudnn) 0.414 ms (166 TF) → 手写版 95%

验收线: 教学版 >= SDPA 最优后端 50% 合格、>85% 优秀 → 实测 95-127%，全部"优秀"

(同一张空闲卡的另一次独立运行实测 105.4%~163.4%，该次为 4090 D、与主表不同卡；计时的"独占 vs 共享/同卡"是公平性变量，见教程 4.4)

> 教学版只做前向（不物化 logsumexp、无 backward），SDPA 要为 autograd 额外写

> LSE，小形状下手写版可能反超；比较内核快慢须同卡同负载（独占/共享数字会整体漂移）。详见

> [05 章](05_flash_attention.md)的实测小节。

课后作业

每章末尾有思考题（<details> 折叠答案）。全部学完后：

[Assignment 9](../../assignments/assignment_9/)

(题 1-4 纯 CPU 可完成：索引数学 / 行列主序 / tiling 账本 / GFLOPS 报告；题 5 是 Triton 实战)

相关资源

- [infatoshi/cuda-course](https://github.com/infatoshi/cuda-course) — 本部分参考的项目 (FreeCodeCamp)
- Simon Boehm：[How to Optimize a CUDA Matmul Kernel to cuBLAS in 1 Hour](https://siboehm.com/articles/22/CUDA-MMM) — matmul 阶梯的出处
- [siboehm/SGEMM_CUDA](https://github.com/siboehm/SGEMM_CUDA) — 阶梯完整代码
- [karpathy/llm.c](https://github.com/karpathy/llm.c) — 本课程的"毕业读物"（纯 C/CUDA 训练 GPT-2）
- [NVIDIA CUDA C++ Programming Guide](https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html) — 官方手册
- PMPP (Programming Massively Parallel Processors) — GPU 编程圣经
- [triton-lang/triton](https://github.com/triton-lang/triton) — Triton 官方仓库 + tutorials (fused attention 等)
- [GPUMODE](https://www.youtube.com/@GPUMODE) — 每周 GPU 内核讲座 (原课程推荐)

[← 上一章：Part 8 后训练全流程](../../Part8_post_training/tutorial/README.md) | [下一章：Part 10 分布式训练 →](../../Part10_distributed/tutorial/README.md)

01_gpu_and_first_kernel

01 — GPU 架构与第一个 CUDA 内核

- > 前八课里，`tensor @ tensor` 一直是个黑盒。从这章起我们打开它：GPU 到底是什么、
- > 怎么把一句 `c[i] = a[i] + b[i]` 变成一百万个并行线程，以及为什么 GPU 能快 10 倍以上。

学习目标

完成本章后，你将能够：

- 理解 CPU 与 GPU 的设计哲学差异（低延迟 vs 高吞吐）及深度学习为何选中后者
- 画出 grid/block/thread/warp 三级执行层级，并说出它们到硬件（core/SM/GPU）的映射
- 手写 1D/2D 全局索引公式，解释边界守卫 `if (i < n)` 为什么必须存在
- 复述 CUDA 程序五步曲（分配→拷入→启动→同步→拷回）并跑通第一个内核
- 验证 GPU 内核的正确性：先写 CPU 参照版，再逐元素对照

前置知识

必须掌握：

- 够用的 C：能读懂 `for` 循环和 `a[i]` 数组下标即可——本章有"够用的 C"速成节（指针/数组/宏/编译，10 分钟版），零基础也能跟上

建议掌握：

- Part 6：self-attention 里 `q @ k^T`、`softmax` 这些"算子"的概念（我们只会用到"算子"这个词，不需要推导）
- Part 7 03 章：Flash Attention、KV Cache 被提过是"内核级优化"——本章之后你就懂这个词

可选：

- Part 8：bf16 autocast——混精度为什么省显存/加速，学完 Part 9 会更具体

> 没有 GPU？本章的概念全部可以纸上学习；`scripts/` 里的 `.cu` 需要

> NVIDIA GPU + `nvcc` 编译，可以放到 [Google Colab](https://colab.research.google.com/)（免费 T4）

> 或 Kaggle（每周 30h P100）上跑。作业 9 的题 1-4 纯 CPU 可完成。

环境自检（对应原课程 02_Setup）

```
`bash
nvidia-smi # 能看到显卡 → 驱动 OK
nvcc -V # 能看到 release 11.x/12.x → 编译器 OK（没有的话见本课 README）
python -c "import torch; print(torch.cuda.is_available())" # True → PyTorch 侧 OK`
```

三者互相独立：`nvidia-smi` 有但 `nvcc` 没有 = 只装了驱动没装 CUDA Toolkit；
`nvcc` 有但 `torch` 说 `False` = `torch` 装成了 CPU 版。这三种坑都常见。

深度学习生态里的 CUDA（对应原课程 01 课）

你在 Part 1-8 写的每一行 PyTorch，实际执行的路径是：

```
`你写的 Python: out = x @ W + b
↓ ATen (PyTorch 的 C++ 算子库)
库调用: cublasGemm(...) / conv kernel / ...
↓ CUDA Runtime / 驱动
硬件: GPU 上的几万个线程同时做乘加`
```

- CUDA 是 NVIDIA 的并行计算平台：一套 C/C++ 扩展语法 + 编译器（`nvcc`）+ 运行时。

- cuBLAS / cuDNN 是 NVIDIA 的预写内核库（矩阵乘 / 卷积等），PyTorch 的大部分算子直接调它们。
- 所以"PyTorch 慢"几乎从来不是 Python 慢，而是内核选择/访存模式的问题——这就是为什么有时候手写一个融合内核能快 10 倍（Part 8 的 GRPO 采样循环就是典型受益者）。

原课程把 CUDA 课程的目标定为"为读懂 Karpathy 的 llm.c 打基础"——llm.c 就是用纯 CUDA+C++ 复刻 GPT-2 训练，不依赖 PyTorch。学完本课你会具备读它的第一块拼图。

CPU vs GPU：两种哲学（对应原课程 04 课）

	CPU	GPU
设计目标	**低延迟**：尽快算完这一件事	**高吞吐**：同时算完一大堆事
核心数	几个~几十个大核，主频高	几千~上万个"小核"（4090：16384 个 CUDA core）
每核能力	分支预测、乱序执行、大缓存	极简，靠数量取胜
擅长	逻辑复杂、分支多的串行任务	大规模的**规则**并行计算
典型弱项	一万个数相加（逐个来太慢）	`if/else` 密集的串行逻辑

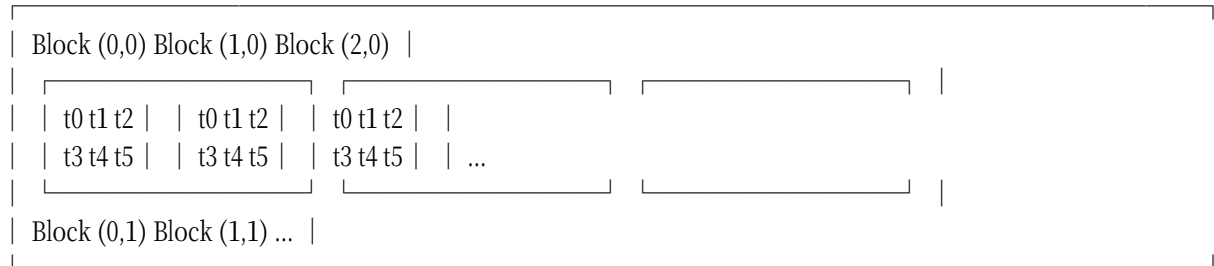
- > GPU 不是"更快的 CPU"，而是完全不同的机器。把 CPU 比作 4 位教授，GPU 是 16384 个小学生：
- > 教授解奥数题完胜；但让 16384 个小学生每人算一页乘法口诀，瞬间完成。
- > 深度学习的本质 = 海量矩阵乘加（小学生型任务）→ 这就是 GPU 统治 DL 的原因。

△ 数据要来回搬运：CPU 内存（主机内存）和 GPU 显存是两个世界，中间隔一条 PCIe 总线。数据搬运慢、计算快，所以"小任务上 GPU 反而更慢"——拷贝开销吃掉了收益。这是后面 streams 一节（03 章）要解决的问题。

GPU 的执行层级（对应原课程 05 课）

CUDA 把并行组织成三层，这是整门课最重要的一张图：

Grid（网格）= 一次内核启动的全部线程



硬件映射：

thread → CUDA core（最小编排单位）

block → SM（Streaming Multiprocessor，一个 SM 同时驻留多个 block）

grid → 整块 GPU

再加一个必须知道的概念——warp：

- warp = 32 个连续编号的线程组成的"小队"，它们锁步（lock-step）执行同一条指令。
- SM 真正的调度单位是 warp，不是单个线程。block_size 必须取 32 的倍数（128/256/512 常见），否则最后一个 warp 有空座，纯浪费。
- △ 同一 warp 里线程走不同分支（如 `if (i % 2)`）时，两个分支要串行各跑一遍

(warp divergence, 分支发散), 并行度减半。内核里少写依赖数据的分支。

够用的 C (对应原课程 03 课, 10 分钟版)

概念	代码	一句话解释
指针	<code>'float *p'</code>	p 存的是"地址", <code>'p[i]'</code> 等价于"从 p 指向的位置数第 i 个"
数组即指针	<code>'A[i * K + l]'</code>	二维数组按行摊平成一维后, <code>'(i,l)'</code> 元素在第 <code>'i*K+l'</code> 格 (row-major)
动态内存	<code>'malloc / free'</code>	主机内存版; GPU 显存用 <code>'cudaMalloc / cudaFree'</code>
宏	<code>'#define N 1000000'</code>	编译前文本替换, 定义常量
编译	<code>'nvcc a.cu -o a'</code>	<code>nvcc = gcc + CUDA 编译器</code> : <code>'cu'</code> 里的 CPU 代码交给 gcc, GPU 代码 (kernel) 编译成 PTX/SASS

`nvcc` 的编译流水线 (读 `llm.c` / PyTorch 编译报错时你会认得这些词) :

`.cu` → (`nvcc` 前端) → PTX (虚拟汇编) → (`ptxas`) → SASS (具体架构的机器码)。

`nvcc -ptx a.cu` 可以导出 PTX 人工阅读——原课程 07 课用它验证循环是否被展开。

第一个内核: vector add (跑 [scripts/01_vector_add.cu](../scripts/01_vector_add.cu))

任务: $c[i] = a[i] + b[i]$, i 取 $0..999999$ 。CPU 版就是 for 循环; GPU 版分两半:

① 内核 (在 GPU 上跑的函数) :

```
`cuda
__global__ void vector_add_gpu(float a, float b, float *c, int n) {
int i = blockIdx.x * blockDim.x + threadIdx.x; // 我是谁?
if (i < n) { // 边界守卫
c[i] = a[i] + b[i];
}
}
```

• `__global__` 声明"从 CPU 启动、在 GPU 执行"的函数 (另两个兄弟: `__device__` 只在 GPU 内调用、`__host__` 只在 CPU)。

• 每个线程执行同一份代码 (SIMT), 靠"我是谁"算出各自处理的下标——这就是 CUDA 的核心思想。

• 全局索引公式 $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$:

把它读成"第几班 × 每班人数 + 班内学号"。比如 `blockDim=256` 时, `block 3` 的 `17` 号线程是 $i = 3 * 256 + 17 = 785$ 。

• `if (i < n)` 边界守卫: 线程总数要向上取整到 `block` 的倍数 ($3906.25 \rightarrow 3907$ 个 `block`), 多出来的线程必须拦住, 否则越界读写显存。

② 主函数 (CPU 侧的五步曲) :

```
`cuda
// 1. 分配显存 2. 拷输入到显存
cudaMalloc(&d_a, size); cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
// 3. 启动内核: <<<block数, 每block线程数>>>
vector_add_gpu<<<3907, 256>>>>(d_a, d_b, d_c, N);
// 4. 等 GPU 真正做完 (启动是异步的!) 5. 拷结果回来
cudaDeviceSynchronize(); cudaMemcpy(h_c_gpu, d_c, size, cudaMemcpyDeviceToHost);
```

- △ 内核启动是异步的：<<<...>>> 这一行瞬间返回，GPU 在后台慢慢算。
不 `cudaDeviceSynchronize()` 就去读结果，读到的是没算完的垃圾。PyTorch 用户对应的坑：
忘了 `torch.cuda.synchronize()` 就计时，测出的"耗时"永远是 0.001ms。
- `h_/d_` 前缀是社区惯例：host（主机）/device（设备）。

实测输出（4090，N=100 万）：

```
CPU avg: 0.230 ms
GPU avg: 0.019 ms (kernel only, no memcpy)
Speedup: 12.2x
Verification: CORRECT (0 errors)
```

- > 验证方法论（原课程反复强调）：先写 CPU 版当标准答案，GPU 算完逐元素对比。
- > GPU 代码错了不会报错，只会安静地给出错答案——CPU 参照物是唯一可靠的防线。
- > 这也是我们课程 Part 1 的老传统：计数版 bigram 对照神经网络版。

线程层级实操（跑 [scripts/02_thread_hierarchy.cu](../scripts/02_thread_hierarchy.cu)）

这个脚本让每个线程打印自己的坐标：

```
=== whoami: grid(2,2) x block(2,2), 16 threads, order NOT guaranteed ===
block(1,1,0) thread(0,0,0)
block(1,1,0) thread(1,0,0)
block(0,1,0) thread(0,0,0)
block(0,0,0) thread(1,1,0)
...（顺序每次都不一样！）
```

- 打印顺序乱：GPU 不保证 block 的启动顺序——16 个线程真的在并行执行。
这是学生第一次"亲眼见到"并行的地方。
- 2D/3D 启动：`dim3 grid(2,2), block(2,2)`，索引公式对 x/y/z 各算一遍。
- 同一份数据，用 1D 启动和 2D 启动算 square，结果必须一致（脚本验证了这一点）——
"1D/2D/3D"只是给线程编号的方式，数据本身永远是一维线性内存。
- 2D 的价值在 matmul：row/col 两个坐标天然对应输出矩阵的两个维度（下一章主角）。

学完本部分你能...

- 画出 grid/block/thread/warp 层级，说出它们到硬件（core/SM/GPU）的映射
- 手写全局索引公式（1D/2D），解释为什么需要边界守卫
- 完整说出 CUDA 程序五步曲：分配显存 → 拷入 → 启动内核 → 同步 → 拷回
- 解释"为什么 GPU 快"和"什么时候 GPU 反而慢"（搬运开销、分支密集）
- 用 CPU 参照实现验证 GPU 内核的正确性

课后练习

<details>

<summary>Q1: 为什么 blockDim 一般取 128/256/512，而不是 100？</summary>

A: warp = 32 线程锁步执行。SM 按 warp 分配调度槽位， $100 = 3 \text{ 个完整 warp} + 4 \text{ 个空座}$ ；这 4 个线程所在的 warp 只用了一半多一点的 lanes，浪费硬件资源。取 32 的倍数让每个 warp 都满员。

</details>

<details>

<summary>Q2: 把 vector add 的 `if (i < n)` 删掉，N 恰好等于线程总数时，程序对吗？值得删吗？</summary>

A: 这次恰好对（没有多余线程），但这是脆弱的巧合——N 一变就崩。守卫只花一次整数比较，相比内存读写几乎免费，永远保留。安全 > 一点点微小的性能。

</details>

<details>

<summary>Q3: 一个 block 最多 1024 个线程。想用 4096 个线程怎么办？为什么要限制？</summary>

A: 拆成 4 个 block ($4096/1024$)，block 之间由 GPU 调度到各个 SM 并行执行。

限制 block 大小是因为一个 block 的所有线程必须驻留在同一个 SM 上（共享同一块 SMEM、共用 barrier 同步），SM 的寄存器/SMEM 数量物理上装不下太多线程。

</details>

课后作业

完成本章后，去 Assignment 9 完成题 1（索引数学）和题 2（行/列主序）：

[Assignment 9](../../assignments/assignment_9/)

下一步

会写"最简单的内核"了。但深度学习的核心是 matmul——下一章我们写它，测出第一份 GFLOPS 报告，然后一梯一级把它从 500 GFLOPS 优化到 8000+，并搞清楚每一级优化到底在省什么。

[02 — matmul 优化阶梯：从 naive 到 cuBLAS](02_matmul_optimization.md)

02_matmul_optimization

02 — matmul 优化阶梯：从 naive 到 cuBLAS

- > 深度学习 90% 以上的浮点运算花在 matmul 上。本章把"最朴素能跑"的 GPU matmul
- > 当起点，走一遍业界标准的优化阶梯，每一级用实测 GFLOPS 验证收益，
- > 并回答一个面试高频问题：为什么 GPU 快、快在哪、瓶颈到底是什么。

学习目标

完成本章后，你将能够：

- 判断一个内核卡在哪堵墙：用算术强度区分 compute-bound 与 memory-bound
- 解释合并访存 (coalescing) 为什么值 9 倍速度，并检查一个内核的 warp 访问模式
- 写出 naive matmul 与 SMEM tiling 版本，说出优化阶梯每一级"省的是什么"
- 实测 GFLOPS 并与 cuBLAS 对比，说明差距的构成（向量化 / autotune / Tensor Core）
- 迁移：把 Flash Attention / KV Cache / bf16 的加速原理用"内存墙"语言重新讲一遍

前置知识

必须掌握：

- 01 章：grid/block/thread 层级、全局索引公式、CUDA 五步曲、2D 启动

建议掌握：

- Part 1-6：x@W 在训练里出现的频率（每个 Linear 层、每个 attention 的 q/k/v 投影）
- Part 7 03 章：提过 Flash Attention 是"内核级优化"——本章给出它的直觉

可选：

- [Part 6 教程](../Part6_transformer/tutorial/README.md)：Transformer / attention——知道"q@k^T 是个算子"这个层面就够了（[Part 9 README](README.md) 前置知识的展开）

优化阶梯的出处：Simon Boehm 的博客

[How to Optimize a CUDA Matmul Kernel to cuBLAS in 1 Hour](https://siboehm.com/articles/22/CUDA-MMM)（cuda-course 07 课的主线材料），我们沿用它的 kernel 1→5 命名。

先跑基准：naive

matmul ([scripts/03_naive_matmul.cu](../scripts/03_naive_matmul.cu))

每个 GPU 线程负责输出矩阵 C 的一个元素：先算出自己的 (row, col)，再沿 K 做点积。

```
`cuda
__global__ void matmul_gpu_naive(const float A, const float B, float *C,
int m, int k, int n) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (row < m && col < n) {
float sum = 0.0f;
for (int l = 0; l < k; l++)
sum += A[row * k + l] B[l * n + col]; // 行主序：A[i][k]=A[ik+k]...
C[row * n + col] = sum;
}
}
```

实测（4090, 512×512×512, fp32）：

```
`
GPU naive : 0.057 ms -> 4697.8 GFLOPS
CPU : 52 ms (one shot)
Ideal arithmetic intensity (each byte read once): 85.33 FLOP/byte
-> above 4090 roofline (~82 FLOP/byte): compute-bound in theory;
naive re-reads A row + B col per output (no data reuse),
effective intensity 0.25 FLOP/byte -> memory-bound! (see script 04)
```

CPU 要 52ms, GPU 0.057ms——快了 900 倍。但先别高兴：这离这块卡的潜力差着 4-5 倍。最后四行是关键——注意 85.33 是按最少必读字节算的理想渐进强度（FLOP:byte = 2N:8, 每字节只读一次），它已高于 4090 屋顶线（~82 FLOP/byte），

说明 matmul 这个算法理论上该是 compute-bound；但 naive 实现不复用数据，每个输出都重读整行 A + 整列 B，实际强度只有 ~ 0.25 FLOP/byte——这才是内存墙，也是脚本 04 的 SMEM/寄存器复用要解决的东西。

核心概念：算力墙 vs 内存墙（roofline：FLOPS 与带宽构成的极限包络图）

每个内核都在两种瓶颈中占一种：

算力墙（compute-bound）：计算单元在满转，数据早就备好
→ 提升 = 更多 FLOPS（Tensor Core、更精密的指令）

内存墙（memory-bound）：计算单元大部分时间在【等数据】从内存赶来
→ 提升 = 更少的内存读写（tiling、融合、缓存）

判断方法叫算术强度（arithmetic intensity）：

算术强度 = 总 FLOPs / 总内存字节数

naive matmul: $2MNK$ FLOPs，但要读 $2MNK$ 次数据（每个输出沿 K 读 A 行+B 列）
 ≈ 1 FLOP : 1 次全局读 → 强度太低 → 内存墙

- LLM 推理（生成阶段）几乎总是 memory-bound：每生成一个 token 要把全部权重读一遍，计算只占一小部分时间。这就是为什么 Part 7 的 KV Cache 有效（避免重复算）、为什么量化有效（读的每个数变小）、为什么 batching 能提升吞吐（一次权重读喂多个请求）。
- △ 面试里说"GPU 快是因为核多"只对了一半；**完整的说法是：核多 + 分级内存 + 海量线程把访存延迟藏起来**。接下来三招全在围绕"内存"做文章。

优化前必修：合并访存（coalescing）

一个 warp（32 线程）同时发起内存请求时，如果它们的地址连续，硬件把它们合并成一次内存事务（128 字节）；如果地址分散，就拆成 32 次独立事务——带宽浪费 32 倍。

naive 版里藏着两种访问模式（warp = 同一行的 32 个相邻 col 线程）：

$A[\text{row} * k + l]$ ：32 个线程读【同一个】地址 → 广播，免费 ✓
 $B[l * n + \text{col}]$ ：col 相邻 → 地址相邻，合并成 1 次事务 ✓

我们脚本 03 的线程映射（ $x \rightarrow \text{col}$ ）已经是合并的。脚本 04 的 L1 故意把映射写坏（ $x \rightarrow \text{row}$ ），让 B 的读取跨行跳跃：

L1 uncoalesced 553.6 GFLOPS ← 坏映射，比 L2 慢 ~ 9 倍
L2 coalesced 4844.9 GFLOPS ← 同样的计算量，只是"读的方式"对了

- > 同一个算法，仅仅是线程到数据的映射方式不同，差 9 倍——这是本课最震撼的一组数字，
- > 也是"GPU 编程 = 访存编程"的最好证据。

优化阶梯 ([scripts/04_matmul_tiled.cu](../scripts/04_matmul_tiled.cu))

阶梯全貌 (4090 实测, 512³ fp32) :

kernel time(ms) GFLOPS 优化的是哪堵墙

L1 uncoalesced 0.485 553.6 (对照组: 访存做错)

L2 coalesced 0.055 4844.9 合并访存

L3 smem tile 0.045 5905.4 shared memory 复用

L4 1D blocktile 0.039 6967.5 寄存器复用 (每线程 8 输出)

L5 2D blocktile 0.031 8795.2 寄存器复用最大化 (4x4 微tile)

cuBLAS (脚本 06) 0.012 22163.1 Tensor Core + autotune + 向量化

L3: shared memory (SMEM) —— 每线程仍 1 个输出

观察浪费: 输出 tile 的所有元素都依赖 A 的同一批行、B 的同一批列。

naive 让每个元素各自去全局内存读 → 同一份数据被重复读 N 次。

做法: 每个 block 把需要的 A/B 小块 (tile) 一次性搬进 SMEM (每 SM 独享的高速片上内存, ~100KB 级, 延迟接近寄存器), 之后整个 K 循环都在 SMEM 里进行:

```
`cuda
__shared__ float As[BM][BK]; // block 内所有线程共享
__shared__ float Bs[BK][BN];
for (int t = 0; t < k; t += BK) {
    // 合作搬运: block 里每线程搬 1 个 A 元素 + 1 个 B 元素
    As[ty][tx] = A[aRow * k + aCol];
    Bs[ty][tx] = B[bRow * n + bCol];
    __syncthreads(); // 等 tile 全部落位 (block 内屏障!)
    for (int l = 0; l < BK; l++)
        sum += As[ty][l] * Bs[l][tx];
    __syncthreads(); // 算完再搬下一块, 防止有人提前改写
}
```

- `__syncthreads()` 是 block 内的屏障: 所有线程到齐才放行。它是 block 这个抽象存在的根本原因——只有同 block 的线程能通信 (SMEM + barrier), 跨 block 只能等内核结束。

- 这就是 Part 7 提过的 Flash Attention 的核心思想: 把 Q/K/V 分块 (tile) 搬进 SMEM, 在片上算完一部分 attention 再换下一块, 全局内存里从不出现巨大的注意力矩阵。

FA 的额外难点是 softmax 需要"整行"信息——解法叫 online softmax: 维护运行最大值 m 与已累加的和, 新块进来时把历史累加和按 $\exp(m_{\text{old}} - m_{\text{new}})$ 重缩放再继续累加 (详见 FlashAttention 论文 § 3.1)。

> 术语小注: bank=SMEM 按 4 字节切成的 32 个独立读取通道; 同一 warp 的两个线程
> 恰好访问同一 bank 的不同地址时串行化 (bank conflict), 吞吐打折。

L4: 1D block tiling —— 寄存器接力

L3 之后 SMEM 读成了新瓶颈。解法: 让每个线程一次算 8 个输出 (同一列相邻 8 行),

结果全存寄存器（比 SMEM 还快一个量级）：

读 1 次 Bs[] [tx]（SMEM）→ 喂给 8 次乘加 # SMEM 读次数 ÷ 8
sums[0..7] 活在寄存器里，最后一次性写回

L5：2D block tiling——把复用推满

每线程算 4×4 = 16 个输出的微 tile，行方向和列方向的 SMEM 读都被摊薄。
这一版的结构（块级 tile → 线程级微 tile → 寄存器累积）**就是 cuBLAS/CUTLASS 高性能 GEMM 的骨架**。

通向 cuBLAS 还差什么（原课程 07 课的延伸层）

手段	一句话
向量化访存	`float4` 一次搬 128 bit，减少指令数（原课程 07 课 unrolling_example.cu）
`#pragma unroll`	展开循环，让编译器排更多指令填等待空隙（可用 `nvcc -ptx` 验证是否生效）
Autotuning	BM/BN/BK/TM/TN 的最优组合随 GPU 架构变化，grid search 自动选（siboehm 用脚本搜参数）
Double buffering	搬下一块 tile 与算当前 tile 重叠（异步拷贝）
Tensor Core	专用矩阵乘加单元（fp16/bf16/tf32 输入），一次算 4×4 矩阵块——cuBLAS 快的真正大头

- > occupancy（占用率）这个词在原课程 07 课出现过：SM 上活跃 warp 数 ÷ 最大可容纳
- > warp 数。寄存器太多、SMEM 太大的 kernel 会降低 occupancy——优化常常是三者间的权衡，
- > `nvcc -Xptxas -v` 能看到每个内核的寄存器用量。

和我们课程的关系（面试向）

Part 9 概念	前面课程出现的位置	面试问法
memory-bound	Part 7 KV Cache、Part 8 推理加速	"LLM 推理为什么是 memory-bound？怎么优化？"
tiling / SMEM	Part 7 Flash Attention 提及	"Flash Attention 为什么快？省了什么？"
bf16 / Tensor Core	Part 8 autocast(bf16)	"混合精度训练为什么几乎不掉点还更快？"
GFLOPS / 算术强度	本课新引入	"给你一个 kernel，你怎么判断它该往哪个方向优化？"

学完本部分你能...

- 写出 naive matmul 内核，并算出它的算术强度、判断瓶颈类型
- 解释合并访存为什么值 9 倍速度，检查一个内核的 warp 访问模式
- 说出优化阶梯每一级"省的是什么"：coalesced→事务数、SMEM→重复读、寄存器→SMEM 读
- 把 Flash Attention / KV Cache / bf16 的加速原理用"内存墙"语言重新讲一遍
- 用 GFLOPS 对比手写内核与 cuBLAS，说出差距的构成

课后练习

<details>

<summary>Q1: 512³ 的矩阵只有 1MB×3，全塞得进 4090 的 72MB L2 cache——我们的实测阶梯

里 L2→L5 的提升因此偏小。换成 4096^3 (192MB×3) 重测，哪几级提升会变大？</summary>
A: 全部以"减少全局读"为卖点的级 (L3/L4/L5) 差距会拉大：小矩阵时 L2 把 naive 的重复读都缓存住了，tiling 的优势被掩盖；大矩阵重复读全打到 HBM，SMEM/寄存器复用才是硬收益。教训：benchmark 要用目标规模测（原课程反复强调 verify + realistic size）。</details>

<details>
<summary>Q2: 为什么 block tile 取 64×64 、BK 取 8 这类数？取 96×96 行不行？</summary>
A: 约束有一堆：每 block 线程数 ≤ 1024 ；SMEM 用量 BMBK + BKBN 个 float 要 $\leq$ 每 SM 上限；寄存器用量（每线程 TM*TN 个累加器）不能把 occupancy 压死；BK 取 8/16/32 与共享内存 bank 数（32）和 warp 大小对齐友好。 96×96 未必不行——**没有普适最优解，所以工业界用 autotuning 按显卡实测搜参数**（这正是 siboehm 的 autotune 一章做的事）。</details>

<details>
<summary>Q3: attention 的 $Q@K^T$ 是 $(T,d)@(d,T)$ ——它在 LLM 里为什么也是"大 matmul"？seq=4096、d=128、heads=32 时 FLOPs 是多少？</summary>
A: 每个 head 一次 $Q@K^T$ 是 $T \times T \times d$ 的 matmul ($2TT*d$ FLOPs)，32 个 head 并行。 $4096^2 \times 128 \times 2 \times 32 \approx 137$ GFLOPs——一次 forward 仅这一项就上百 GFLOPs；再加上 V 加权和（同量级）和 FFN（通常是 attention 的 2-4 倍），单层单次 forward 就是数百 GFLOPs。所以"内核效率"直接等于"训练账单"。</details>

课后作业

完成本章后，去 Assignment 9 完成题 3（tiling 访存账本）和题 4（GFLOPS 报告）：

[Assignment 9](../../assignments/assignment_9/)

下一步

内核会写了、会优化了，但还没回答：怎么测量（profiling）、怎么协作（atomics / streams）、什么时候不该自己写（cuBLAS / cuDNN）。下一章补齐这三块工程拼图。

[03 — Profiling、Atomics、Streams 与 CUDA 库](03_profiling_and_cuda_apis.md)

03_profiling_and_cuda_apis

03 — Profiling、Atomics、Streams 与 CUDA 库

- > 会写、会优化之后，还差三块工程拼图：怎么测量内核时间花在哪（profiling）、
- > 线程之间怎么协作（atomics / 归约）、任务之间怎么重叠（streams），
- > 以及什么时候不该自己写——直接调 cuBLAS / cuDNN。

学习目标

完成本章后，你将能够：

- 使用 nsys / ncu 两件工具，从 SOL 表判断内核卡在 Compute 还是 Memory
- 写出 树形归约内核，解释"原子操作为什么对但慢"以及分层归约快 77 倍的原因
- 解释 stream 的重叠原理，并复述我们"重叠反而更慢"的实测教训
- 推导 行主序/列主序恒等式，手动写出 cuBLAS 的参数组合
- 决策：什么场景该直接调库、什么场景才值得手写内核

前置知识

必须掌握：

- 02 章：内存墙/算力墙、warp、SMEM、__syncthreads()

建议掌握：

- Part 3：诊断工具的思路（先测量、再下结论）——本章就是 GPU 版的"给内核做体检"

可选：

- [Nsight Systems 文档](https://docs.nvidia.com/nsight-systems/) / [Nsight Compute 文档](https://docs.nvidia.com/nsight-compute/)——两件 profiling 工具的官方手册，用到再查

Profiling：先测量，再优化（对应原课程 05 课 03 节）

手搓计时（cudaEvent / clock_gettime）只能给你总时间。想知道时间花在哪，用 NVIDIA 的两件工具：

工具	干什么	一句话用法
Nsight Systems (nsys)	时间线全景：内核、memcpy、CPU-GPU 交替	`nsys profile -o out ./bin/03_naive_matmul` → 看"谁在等谁"
Nsight Compute (ncu)	单个内核的深入剖析：SM 占用率、内存吞吐、瓶颈判定	`ncu --set full ./bin/04_matmul_tiled` → 看 SOL（Speed Of Light）表

实测 nsys：数据搬运比内核还贵（4090，driver 550.120，CUDA 12.4）

```
`bash
nsys profile -o /tmp/p9_03 ./bin/03_naive_matmul # 采集：生成 .nsys-rep
nsys stats --report cuda_gpu_kern_sum \
--report cuda_gpu_mem_time_sum /tmp/p9_03.nsys-rep
`

** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):
Time (%) Total Time (ns) Instances Avg (ns) Med (ns) Min (ns) Max (ns) Name

100.0 326,795 6 54,465.8 54,178.0 54,114 55,714 matmul_gpu_naive(...)

** CUDA GPU MemOps Summary (by Time) (cuda_gpu_mem_time_sum):
Time (%) Total Time (ns) Count Avg (ns) Operation

63.4 100,452 2 50,226.0 [CUDA memcpy Host-to-Device]
36.6 57,922 1 57,922.0 [CUDA memcpy Device-to-Host]
```

> 读表：6 次内核（warm-up + 5 次计时）每次 $\sim 54 \mu s$ ；而 H2D 拷贝 $2 \times 50 \mu s$ + D2H 拷贝 $> 58 \mu s \approx 158 \mu s$ ——搬运是内核的 3 倍。这就是 01 章"小任务上 GPU 反而慢"的定量版：
> 脚本 03 计时只算内核、不含拷贝，端到端看时间线才知道拷贝占大头。

实测 ncu：SOL 表直接指认瓶颈（同机）

```
`bash
```

⚠ 普通用户直接跑会报 ERR_NVGPUCTRPERM（GPU 性能计数器默认仅管理员可用）：

sudo 运行，或让管理员设
NVreg_RestrictProfilingToAdminUsers=0。

nsys

不碰计数器，普通用户即可跑（上面就是普通用户跑的）。

```
sudo ncu --section SpeedOfLight --launch-skip 2 --launch-count 1 ./bin/03_naive_matmul
```

```
matmul_gpu_naive(...) (16,16,1)x(32,32,1), Context 1, Stream 7, Device 0, CC 8.9  
Section: GPU Speed Of Light Throughput  
Metric Name Metric Unit Metric Value
```

```
Memory Throughput % 92.01  
DRAM Throughput % 3.36  
Duration usecond 64.58  
L1/TEX Cache Throughput % 94.15  
L2 Cache Throughput % 15.06  
Compute (SM) Throughput % 92.01
```

> 读表：naive 的 L1/TEX 管道 94% 打满——2MNK 次重复读全砸在缓存管线上；
> 而 DRAM 只有 3%（ 512^3 三块矩阵 $1MB \times 3$ ，全装在 4090 的 72MB L2 里）。
> 这就是 02 章访存账本的实测版："memory-bound" 绑的是哪级存储，SOL 表一目了然。

再看优化阶梯尽头的 L5（`--kernel-name regex:matmul_2d`）：

```
matmul_2d(...) (8,8,1)x(16,16,1), Device 0, CC 8.9  
Memory Throughput 16.42% Compute (SM) Throughput 12.50%  
OPT This kernel grid is too small ... only 0.1 full waves across all SMs
```

两堵墙都没撞（16%/12%）——ncu 的 OPT 提示点破真相： 512^3 只有 64 个 block，4090 的 128 个 SM 吃不饱（0.1 waves）。先扩大规模，再谈优化——这是"benchmark 要用目标规模测"的 profiling 版证据。

原课程 03 Profiling 课还演示了 NVTX：在代码里插 `nvtxRangePush("forward") /`

`nvtxRangePop()`，时间线上就有彩色的阶段标记，一眼分清 forward/backward/数据搬运。
(PyTorch 的 `torch.profiler` 底层就是这套设施 + CUPTI。)

- > profiling 的纪律（呼应 Part 3 的"诊断→治疗"）：优化前先看数据。
- > ncu 的 SOL 表直接告诉你这个内核卡在 Compute 还是 Memory——对应 02 章的两种墙，
- > 别凭感觉优化。

Atomics：一百万个线程抢一个数 ([scripts/05_atomics_streams.cu](../scripts/05_atomics_streams.cu) Part A)

任务：GPU 上对 100 万个数求和。第一直觉的写法是错的：

```
`cuda
__global__ void sum_naive_wrong(float x, float result) {
int i = blockIdx.x * blockDim.x + threadIdx.x;
result[0] += x[i]; // 读-改-写三步会被别的线程插队 → 竞争条件
}
```

实测它每次输出都不一样（我们跑出过 3.88，正确答案是 500006）。修法是原子操作：

```
`cuda
atomicAdd(result, x[i]); // 读-改-写不可分割；冲突线程被硬件串行化
```

正确了，但 100 万次原子写互相排队，实测 1.396ms——比整个 naive matmul 还慢。
标准解法是分层归约：

```
`
第一层（block 内）：shared memory 树形归约
256 个数两两相加 → 128 → 64 → ... → 1，共  $\log_2(256)=8$  步
（相邻线程加相邻线程，无 bank 冲突；每步 __syncthreads()）
第二层（block 间）：每个 block 的最终和 atomicAdd 到全局
原子操作次数从 1,000,000 降到 3,907（block 数）
```

```
`
atomic : 500004.22 (1.396 ms)
tree + atomic : 500006.62 (0.018 ms) <- 正确且快 77 倍
```

- 这个"先局部、再全局"的模式是 GPU 上一切归约（sum/max/norm）的原型。PyTorch 里 `tensor.sum()` 背后就是这样的多层树形归约内核。
- 排队/竞争这个词在多线程 CPU 编程里也见过（`+=` 不是原子的）——概念完全同构，只是 GPU 的"线程数"大了四个数量级，问题被放大到肉眼可见。

Streams：把"搬运"和"计算"重叠 ([scripts/05_atomics_streams.cu](../scripts/05_atomics_streams.cu) Part B)

默认情况下，你的所有操作排在一条默认流里串行执行：
拷贝 → 算 → 拷回 → 拷下一块 → ……（PCIe 搬运时 GPU 计算单元闲着）。

Stream = 一条独立队列。两条流里的任务可以并行：

```
串行（默认流）:[copy1][calc1][back1][copy2][calc2][back2] ...
两条流交错: stream0: [copy1][calc1] [back1]
stream1: [copy2][calc2][back2] ← copy2 和 calc1 重叠
```

关键 API 只有三个：`cudaStreamCreate(&s)`、启动时把流当第四个参数
`kernel<<<grid, block, 0, s>>>(...)`、`cudaMemcpyAsync(..., s)`（异步版拷贝）。

> △ 演示简化说明：脚本里 4 个 chunk 共用同一块 `d_x` 缓冲——只测时延形态、
> 不校验数值；正式实现应每 chunk 独立缓冲。

△ 诚实的实测教训：我们的脚本里 4 块 1MB 数据双流重叠后是 1.026ms，
反而比串行 0.942ms 慢一点。为什么？① 每块太小，拷贝只有微秒级，
内核更是纳秒级——重叠收益小于 async 调用本身的额外开销；② 现代 GPU 的拷贝引擎
和小内核原本就能部分重叠。streams 真正赢的场景：单块数据量 GB 级、
或"持续输入流式处理"。这条教训本身就是本课要教的东西：直觉必须经过 benchmark 检验
（原课程方法论：always verify, always benchmark）。

cuBLAS：把 matmul 交给库（[scripts/06_cublas_sgemm.cu](../scripts/06_cublas_sgemm.cu)）

手写阶梯的尽头是 8795 GFLOPS（L5），cuBLAS 同一块卡上是 22163 GFLOPS——
还有 2.5 倍差距（Tensor Core、autotune、向量化、double buffering）。
工程上永远直接调库，手写 matmul 是为了"知道库在做什么、什么时候库不是最优"。

cuBLAS 最著名的坑：列主序（column-major）。它是 Fortran 血统，矩阵默认"一列挨一列"存；
而 C/PyTorch 是行主序。我们的脚本用了一个漂亮技巧零成本适配：

关键恒等式：一块"行主序存储"的缓冲区，按列主序去读，读到的正好是它的转置。

要算行主序 $C = A @ B$ ：

列主序视角下，三块缓冲区"读出来"分别是 A^T 、 B^T 、 C^T

而 $C^T = (A @ B)^T = B^T @ A^T$

→ 让 cuBLAS 算 " $B^T @ A^T$ "（指针换序，不搬运任何数据），写回的内存就是行主序 C

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
&alpha, d_B, N, // ← B 的位置，按列主序读 = B^T
d_A, K, // ← A 的位置，按列主序读 = A^T
&beta, d_C, N);
```

实测我们第一次用 OP_T/OP_T 的参数组合，结果全错（max_err=37）——
cuBLAS 的参数写错不报错，只会安静地给你错的答案，CPU 参照验证再一次救场。

- cuBLAS 家族：**cuBLAS**（单卡常规 GEMM）、**cuBLASLt**（带启发式的进阶接口，
可选融合 epilogue——把 bias/激活等附加计算拼进 GEMM 尾部）、**cuBLASXt**（多卡拆分矩阵）。PyTorch 的
@ 最终落到它们。
- cuDNN 同理，是 DL 算子的全家桶（conv/RNN/attention/activation），
原课程 06 课演示了 cuDNN 版 Tanh 和 Conv2d，并与 torch 结果对照。

关于"错误处理"的成人礼

脚本里我们略去了错误检查让代码短一点，但真实工程每个 CUDA 调用都该包一层：

```
`c
#define CUDA_CHECK(call) do {\
    cudaError_t err = (call);\
    if (err != cudaSuccess) {\
        fprintf(stderr, "CUDA error %s at %s:%d\n", \
            cudaGetErrorString(err), __FILE__, __LINE__); exit(1);\
    } } while (0)

CUDA_CHECK(cudaMalloc(&d_a, size));
`
```

△ GPU 错误是异步的：出错的位置可能在很久之后的下一个调用才暴露。

[compute-sanitizer](#) 工具能抓越界访问——调内核 bug 的最后手段。

学完本部分你能...

- 说出 `nsys` 和 `ncu` 各解决什么问题，以及 SOL 表怎么指导优化方向
- 写出树形归约，解释"原子操作为什么对但慢"以及分层归约快 77 倍的原因
- 解释 stream 的重叠原理，并复述我们"重叠反而更慢"的实测教训
- 用行主序/列主序恒等式手动推出 cuBLAS 的参数写法
- 说明什么场景该用库、什么场景才值得手写内核

课后练习

<details>

<summary>Q1: 树形归约里如果 `if (threadIdx.x < s)` 写成 `if (threadIdx.x % 2 == 0)`，除了慢还有什么问题？</summary>

A: 交错配对（thread 0 加 thread 1，thread 2 加 thread 3...）依赖固定的 bank 布局，在 SMEM 上会产生大量 bank conflict，且这种 stride 访问模式和 warp 调度不友好。相邻配对（`x[s] += x[s+idx]`）是标准写法。更细的优化还有 warp shuffle（`__shfl_down_sync`，warp 内不经过 SMEM 直接交换寄存器）——这是 CUB 库干的事。

</details>

<details>

<summary>Q2: PyTorch 里 `x @ W` 你可以指定 `torch.cuda.Stream()` 让它和别的计算并行。结合本节，说说什么情况下值得这么做？</summary>

A: 典型场景：① 梯度交换/数据预取与计算重叠（DDP 的通信流）；② 权重加载/量化转换与推理流水线重叠；③ 多路互不依赖的 batch 并行。不值得的场景：算子间有强依赖（A 的输出是 B 的输入）、内核大到已经吃满 GPU——排队重叠毫无收益还添同步复杂度。

</details>

<details>

<summary>Q3: 为什么 cuBLAS 写错参数不报错？这对你 review 别人的 GPU 代码有什么启示？</summary>

A: cuBLAS 是"参数进、地址上读写"的底层库，它不知道你的"意图"（你要 `A@B` 还是 `B@A`），任何参数组合在它看来都是合法请求。启示：GPU 代码必须配"黄金参照"（CPU 实现或小规模已知答案）+ 形状/数值断言；盲信"能跑 = 对"是 GPU 编程第一大坑。

</details>

课后作业

本章概念在 Assignment 9 的题 3/4 中延伸（访存账本与 GFLOPS 报告是 profiling 的纸面版）：

[Assignment 9](../../assignments/assignment_9/)

下一步

手写 CUDA 你已经走完全程。最后一步：看看工业界怎么让"写内核"这件事变简单——Triton（Python 写内核）和 PyTorch 自定义扩展（把自己的内核接进 PyTorch），并给整个课程画一张"继续往哪走"的地图。

[04 — Triton 与 PyTorch 扩展：通向 llm.c](04_triton_and_extensions.md)

04_triton_and_extensions

04 — Triton 与 PyTorch 扩展：通向 llm.c

> 最后一章回答两个实际问题：① 不写 C++，能不能写内核？（Triton——Part 7 提过的 > Flash Attention、多数现代开源内核的语言）② 自己写的内核怎么接进 PyTorch 训练管线？> 最后给出整个课程的"毕业去向"地图。

学习目标

完成本章后，你将能够：

- 解释 Triton 与 CUDA 的分工：用"编译器替你操心线程"说清各自边界
- 写出带 mask 的 Triton elementwise / 行归约内核，并绕开"嵌套定义"的坑
- 说清 PyTorch 扩展三件套（dispatch / restrict / pybind）各干什么
- 实现 `torch.autograd.Function` 包装，给自定义 CUDA 内核补 backward
- 规划 Part 9 之后的三条进阶路线（学深内核 / 用起来 / 回课程主线）

前置知识

必须掌握：

- 01-03 章：线程层级、SMEM/tiling、合并访存（Triton 帮你管的正是这些）

建议掌握：

- Part 1：softmax 的"减最大值防上溢"技巧（Triton 一节会再见到它）

可选：

- Part 7/8：Flash Attention、KV Cache 出现的位置（本章把它们和内核语言连起来）
- [Triton 官方 tutorials](https://triton-lang.org/main/getting-started/tutorials/index.html)——融合内核的更多实例，想深挖再看

Triton：Python 写内核（对应原课程 08 课）

设计哲学：CUDA vs Triton

原课程 README 里这张对照是精髓：

CUDA = scalar program + blocked threads
你逐线程写代码（标量视角），把线程组织成 block（你来操心）
Triton = blocked program + scalar threads
你按"一块数据"写代码（向量视角），线程怎么划分（编译器替你操心）

	CUDA	Triton
语言	C/C++	Python（装饰器 + `triton.language`）
谁管 tiling/mask/SMEM	你	**编译器**
性能上限	最高（最后 10-20%）	接近（elementwise/reduction 类几乎打平）
写一个 softmax 的代码量	上百行	~20 行
典型用户	NVIDIA、CUTLASS	OpenAI/FlashAttention、Unsloth、torch.compile 的后端

vector add：逐行对照

CUDA ([scripts/07_triton_kernels.py](../scripts/07_triton_kernels.py))

```
`python
@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0) # CUDA: blockIdx.x （这次一个"程序"管一块）
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE) # 这一块的 1024 个下标（向量！）
    mask = offsets < n_elements # CUDA: if (i < n)，但一次判断 1024 个
    x = tl.load(x_ptr + offsets, mask=mask) # load/store 带 mask，越界自动挡
    tl.store(output_ptr + offsets, x + tl.load(y_ptr + offsets, mask=mask), mask=mask)
```

- 没有 threadIdx：你写的是"一块数据怎么算"，编译器自动把它铺到合适的线程/warp 上，并自动做向量化访存（你手写 float4 才能做到的事，这里默认就有）。
- tl.constexpr 的 BLOCK_SIZE 是编译期常量——不同 BLOCK_SIZE 生成不同内核（autotune 的抓手）。
- ⚠ 本章作业（题 5）马上会踩的坑：@triton.jit 函数必须定义在模块顶层，否则 **NameError: tl is not defined**。

softmax：整块加载 + 数值稳定（呼应 Part 1）

```
`python
row = tl.load(row_start_ptr + col_offsets, mask=..., other=-float('inf'))
row_minus_max = row - tl.max(row, axis=0) # Part 1 的老朋友：先减最大值
numerator = tl.exp(row_minus_max)
softmax_output = numerator / tl.sum(numerator, axis=0)
```

一个 program 负责矩阵的一行：整行载入 SMEM（tl.load 自动处理）、片上完成 softmax、一次写回——没有中间的全局内存往返。这就是"内核融合"

(kernel fusion) 的最小样本：torch eager 里 $\text{max} \rightarrow \text{exp} \rightarrow \text{sum} \rightarrow \text{div}$ 四个内核四次显存往返，融合后一遍完成。

实测 (4090)：

```
[vecadd] triton 0.007 ms vs torch 0.004 ms (effective BW ~1.70 TB/s -> memory-bound)
[softmax] triton 0.010 ms vs torch 0.012 ms
```

- > 诚实解读：elementwise 上 Triton 和 torch 内核互有胜负（都是带宽上限附近）；
- > Triton 的价值不在"比 cuBLAS 快"，而在用 1% 的代码量写出"够快"的融合内核——
- > Flash Attention 原始实现、Unsloth、绝大部分 SOTA 开源内核都是 Triton 写的。

PyTorch CUDA 扩展：自己的内核接进训练管线（对应原课程 09 课）

场景：你写好了一个 CUDA 内核（比如算子融合的 activation、自定义 attention），想让 PyTorch 张量直接进出。原课程 09 课用 $x^2 + x + 1$ (polynomial activation) 演示了完整闭环，我们的 [scripts/08_pytorch_extension.py](../scripts/08_pytorch_extension.py) 用 `load_inline()` 现场编译同款：

```
`cpp
template <typename scalar_t> // scalar_t = float 或 double
__global__ void polynomial_activation_kernel(
    const scalar_t* __restrict__ x, // __restrict__: 承诺不重叠 → 放心优化
    scalar_t* __restrict__ output, size_t size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        scalar_t val = x[idx];
        output[idx] = val*val + val + 1;
    }
}

torch::Tensor polynomial_activation_cuda(torch::Tensor x) { // C++ 包装：张量进张量出
    int threads = 1024, blocks = (x.numel() + threads - 1) / threads;
    AT_DISPATCH_FLOATING_TYPES(x.scalar_type(), "...", ([&] { // 按 dtype 实例化模板
        // △ 用 scalar_type(); 老写法 x.type()
        // 在 torch >= 2.4 无法编译
        polynomial_activation_kernel<scalar_t><<<blocks, threads>>>(...);
    }));
    return output;
}
`
```

三个关键件（原课程 README 逐个讲过）：

1. `AT_DISPATCH_FLOATING_TYPES`：同一份内核代码，自动支持 fp32/fp64（泛型分发）。
2. `__restrict__`：向编译器承诺两个指针不重叠，启用激进优化。原课程给了反例：`add_arrays(data, data+3, 7)` 这种重叠调用在 `__restrict__` 下行为未定义。
3. pybind 绑定：`PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) { m.def("polynomial_activation", ...); }`——把 C++ 函数暴露成 Python 函数。（`load_inline()` 的 `functions=[...]` 参数自动生成这段；`setup.py` 打包路线需要手写，见原仓库 09_PyTorch_Extensions/setup.py。）

实测：

```
[speed] custom CUDA : 0.0042 ms (1 个融合内核)
[speed] torch eager : 0.0110 ms (中间张量 + 3 次内核启动)
[grad] 手写 backward 与 (2x+1) 完全一致
```

- $\triangle$ `load_inline()` 编译的是裸函数，没有 autograd 支持——反向传播不会自动工作。

工程做法：`torch.autograd.Function` 包一层，`backward()` 手写（本例一行：`2x+1`）。

这正是理解"PyTorch 算子 = forward 内核 + backward 内核"的最佳练习

（呼应 Part 4 手动反向传播！）。

- $\triangle$ 编译坑（我们在 4090 机上真实撞到）：CUDA 对宿主 gcc 有版本上限（CUDA 11.x 最高 gcc-11），报 `unsupported GNU version` 时给 nvcc 传 `-cbin g++-11`；CUDA 12.1+ 则原生支持 gcc-12/13，无需任何参数。机器上多版本 CUDA 共存时，脚本会自动选版本最高的 `/usr/local/cuda-*`——注意只改 `os.environ["CUDA_HOME"]` 没用：torch 在 `import` 时就把 `CUDA_HOME` 缓存成了模块全局，必须同时覆盖 `torch.utils.cpp_extension.CUDA_HOME`（不动全局环境、不覆盖旧版本——本课开发机就是 11.8 与 12.4 并存）；JIT 编译还需要 `ninja`；产物缓存在 `~/.cache/torch_extensions/`，改了名字/源码不生效时先清缓存。

毕业去向：这门课之后学什么（对应原课程 10/11 课）

原课程的 final project 是"用 CUDA 从零写 MNIST 的 MLP"（仓库只给了架构图，留作练习）。结合我们的课程体系，给三条进阶路线：

路线 A：把内核学深（系统方向）

1. 原课程 Final Project：CUDA 写 MLP forward + 手写 softmax/CE backward（把 Part 4 的手动反传翻译成 CUDA，是极佳的综合练习）
2. PMPP 书（Programming Massively Parallel Processors，GPU 编程圣经）
3. Karpathy 的 `llm.c`——本课开篇说的目标：现在你能读懂它的 `matmul/attention` 内核了
4. CUTLASS：NVIDIA 开源的 GEMM 模板库（02 章阶梯的工业完全体）

路线 B：把内核用起来（Deep Learning 方向）

1. Triton 官方 tutorials：fused-softmax、matmul、flash-attention（把 02 章的直觉落地）
2. `torch.compile`：看看 Inductor 给你的模型生成了什么 Triton 内核
3. GPUMODE 社区（原课程推荐）：每周内核优化讲座 + 竞赛

路线 C：回到课程主线

Part 7/8 训练过的模型，现在你知道：`@` → cuBLAS → Tensor Core；`autocast(bf16)` → 内存带宽减半；KV Cache → 显存里的持久 buffer；DPO/GRPO 的慢 → 采样循环的内核 launch 次数。回去把超参改一改、用 `nsys` 看看时间线——训练器视角和内核视角合璧，才算真正打通"从 tensor 到 SM"。

学完本部分你能...

- 用" Triton=CUDA 的编译器换你操心线程"解释两者的分工
- 写出带 mask 的 Triton elementwise / 行归约内核，并绕开"嵌套定义"的坑
- 说清 PyTorch 扩展三件套（dispatch / restrict / pybind）各干什么
- 用 `torch.autograd.Function` 给自定义内核补 backward
- 给自己画出 Part 9 之后的三条进阶路线

课后练习

<details>

<summary>Q1: Triton 为什么"写不出"cuBLAS 顶级的 matmul？它放弃/隐藏了哪些控制权？</summary>

A: 你无法手控 warp 级原语（shuffle/mma 指令）、寄存器分配、shared memory 的精确布局、double buffering 的调度——这些正是 GEMM 最后 20% 性能的来源（Tensor Core 的 mma 指令排布尤其需要）。Triton 的赌注是：99% 的内核不是 GEMM，把 tiling/mask/向量化自动化收益远大于损失。所以生态分工：GEMM 给 cuBLAS/CUTLASS，长尾融合内核给 Triton。

</details>

<details>

<summary>Q2: 你的自定义 activation 快了 2.6 倍（0.0110→0.0042ms）。为什么 torch eager 慢？如果用 torch.compile 会怎样？</summary>

A: eager 把 $x*x$ 、 $+x$ 、 $+1$ 拆成 3 个内核：各读写一遍 4MB 张量（12MB 流量）+ 3 次启动开销；自定义内核一遍完成（8MB 流量 + 1 次启动）。torch.compile（Inductor）会生成 Triton 融合内核，性能通常接近手写——所以工程顺序是：先 compile，不够再手写。

</details>

<details>

<summary>Q3:（综合 8 课）训练 LLM 时收到"GPU util 只有 40%"的报告，列出至少 4 个可能原因和对应工具。</summary>

A: ① 数据加载慢（CPU 瓶颈）→ nsys 看 GPU 等待间隙，DataLoader 加 workers/prefetch；
② 小 batch/小模型 → kernel launch 开销占比大，batch 拼大或 CUDA Graphs；
③ 频繁 CPU-GPU 同步（.item()/print loss）→ 异步化、少同步；
④ 内存墙算子多（优化器、embedding 查表）→ 融合优化器（fused adamw）、减少 host-device 拷贝。工具链：nsys 时间线定位 → ncu 看单内核 → 改架构/融合。

</details>

课后作业

[Assignment 9](../../assignments/assignment_9/)（题 5：亲手写 Triton softmax）

课程回顾

Part 1-5 用神经网络写人名 → 学会"训练"这件事本身

Part 6 从零搭出 GPT → 学会 Transformer 骨架

Part 7 复现 minimind → 学会现代 LLM 的零件（RoPE/GQA/SwiGLU/MoE）

Part 8 后训练全流程 → 学会 SFT→RM→DPO→PPO→GRPO

Part 9 CUDA 内核 → 学会这一切跑在什么机器上、为什么快、还能更快

下一步见

[← 上一章：Part 8 后训练全流程](../../Part8_post_training/tutorial/README.md)

05_flash_attention

05 — Flash Attention：亲写出毕业内核

> Part 7 复现 minimind 时我们调用过 `F.scaled_dot_product_attention`，02 章讲 matmul
> 阶梯时也预告过它。这一章把 Part 9 的全部家当——02 章的 tiling、07 章的 Triton
> softmax——组装成 Flash Attention 前向内核，并对照 PyTorch SDPA 的四个后端验收
> 正确性与性能。这是本课程的"毕业内核"。

学习目标

完成本章后，你将能够：

- 推导 online softmax 的滚动更新公式 ($m/l/acc$ 三个状态如何跨块传递)
- 手写带 causal mask 与边界处理的 Flash Attention 前向 Triton 内核 (bf16 输入、fp32 累加)
- 解释 FA1→FA2→FA3→FlexAttention→SageAttention 的演进逻辑，以及为什么 4090 用不了 FA3
- 验收自写内核：与 naive 对照数值、与 SDPA 四后端对照吞吐 (≥50% 合格线)

前置知识

必须掌握：

- [02 章 matmul 优化阶梯](02_matmul_optimization.md)：SMEM tiling、算力墙/内存墙——Flash Attention 就是"tiling 思想搬到 attention"，L3 一节已给出它的预告
- [04 章 Triton / 脚本 07](04_triton_and_extensions.md)：`tl.load` 的 mask 用法、"一个 program 管一块数据"的编程模型、softmax 内核里"减最大值防上溢"的技巧 ([脚本 07](../scripts/07_triton_kernels.py) 的 `softmax_kernel` 是本章的直接前作：它对整行做 softmax，本章要解决"整行装不下怎么办")

建议掌握：

- Part 6：attention 公式 $\text{softmax}(QK^T/\sqrt{d})V$ (知道形状怎么变换即可)
- Part 7：`F.scaled_dot_product_attention` 的调用位置 ([Part7 脚本 05](../Part7_minimind/scripts/05_full_model.py) 的 `MiniMindAttention`)、KV Cache 与长上下文的痛点

可选：

- Part 8：bf16 autocast 的位置 (本章内核就是 bf16 输入 + fp32 累加的活例子)

一、为什么需要 Flash Attention

naive 的显存账本

脚本 09 段 1 实测的教科书实现 ([scripts/09_flash_attention_triton.py](../scripts/09_flash_attention_triton.py))：

```
`python
att = (q @ k.transpose(-2, -1)) * scale # (B, H, T, T) <- 第一个巨型中间矩阵
att = att.masked_fill(~keep, -inf) # causal：又物化一份拷贝
p = att.softmax(dim=-1) # (B, H, T, T) <- 第二个
out = p @ v
`
```

实测 (RTX 4090, bf16, B=2, H=8, T=4096, D=64；脚本 09 段 4 的计时协议)：

实现	causal	耗时	附注
naive	否	3.579 ms	物化 2 个 (B,H,T,T) bf16 矩阵, 各 0.5 GB
naive	是	6.198 ms	`masked_fill` 再物化第 3 份, 反而更慢
手写 Triton FA	是	**0.279 ms**	零 (T,T) 中间矩阵
手写 Triton FA	否	**0.436 ms**	

> 以上为共享 GPU 环境实测 (2026-09-02) ; 同一张空闲卡的另一次独立运行 (六场景 105.4%~163.4%, 含 T=1K causal 163.4%——注意该次在 4090 D 上测得, 与主表的 4090 不同卡, 正好示范"跨卡数字不可直接比") ——空闲时手写内核对 SDPA 最优后端的比例更高, > 计时基准的"独占 vs 共享"本身就是一个公平性变量 (见 4.4 的陷阱 4) 。

naive 的三个痛点：

- 1. 显存 $O(T^2)$: 每个 (B,H,T,T) 矩阵在 T=4096 时 0.5 GB ; T 翻倍翻 4 倍。T=8K 时 naive 光中间矩阵就要 ~2 GB $\times$ 3, T=32K 直接爆卡。
- 2. HBM 往返：这些矩阵写回显存再读回来——02 章的语言：内存墙。attention 的计算本身该是 compute-bound, naive 实现却被 $O(T^2)$ 的访存拖住。
- 3. 训练更惨：backward 需要 softmax 的结果 P, 还得把 (T,T) 存着 (或重算) 。

> 类比：算 100 万个数的平均值, 你不会先把 100 万个数全抄到一张超大的纸上
> 再求和——你维护一个"到目前为止的和"滚动更新。Flash Attention 对 softmax 做的
> 就是这件事, 只是多了一个"到目前为止的最大值"来保证数值稳定 (online softmax) 。

> Flash Attention 一句话：把 Q/K/V 切块 (tiling) 搬进片上 SRAM, 用 online
> softmax 让"整行信息"也能分块流式计算——从不物化 (T,T) 矩阵, 显存 $O(T^2) \rightarrow O(T)$,
> HBM 读写减少一个数量级。注意它是精确算法, 不是近似 (与稀疏/线性 attention 相对) 。

演进史：我们要抄的是 FA2 的作业

	论文	关键改进	4090 (SM89) 能用吗
FA1)	[arXiv 2205.14135](https://arxiv.org/abs/2205.14135)	tiling + online softmax, 首次系统做 **IO-aware** 的精确 attention	
(FA2)	[arXiv 2307.08691](https://arxiv.org/abs/2307.08691)	减少非 matmul FLOPs、更优的并行与 warp 分工 (约 2 $\times$ FA1)	(torch SDPA 的 tiling)
(FA3)	[arXiv 2407.08608](https://arxiv.org/abs/2407.08608)	**WGMMA** + 异步流水 + FP8, Hopper 专属	WGMMA 是 **SM90a
	[PyTorch 官方博客](https://pytorch.org/blog/flexattention/)	mask_mod/score_mod $\rightarrow$ torch.compile 生成 Triton 内核	(本章段 5 实测)
	[arXiv 2411.10958](https://arxiv.org/abs/2411.10958)	QK $\wedge$ T 用 INT8、PV 用 **INT4** 量化, 4090 上 ~3 $\times$ FA2	(近似, 推理向)

> △ FA3 与 4090：FA3 的速度来自 Hopper 的 warp 级矩阵指令 WGMMA (SM90a 专属)
> 与 TMA/异步执行。4090 (Ada, SM89) 只有 **mma** 路径, 编译都过不了。所以在我们的卡上,
> "工业天花板"就是 FA2 类实现 (SDPA flash/cudnn 后端) + 量化路线 (SageAttention) 。

> 另外本章内核沿用的在线 softmax 收敛形式最早由 Rabe & Staats ([arXiv 2112.05682](https://arxiv.org/abs/2112.05682)) 给出。

二、online softmax：逐步推导

2.1 从"必须看整行"说起

数值稳定的 softmax (Part 1 / 07 章的老朋友)：

```
\nsoftmax(x)_i = exp(x_i - m) / \sum_j exp(x_j - m), m = max_j x_j\n
```

07 章的 softmax 内核能把整行一次性 `tl.load` 进片上——attention 的行却是 T 个 key 的打分, $T=4096$ 时一行 fp32 有 16 KB, 16 个 $(b,h) \times$ 多个行块根本铺不开, 更别说 (T,T) 全矩阵。分块计算 softmax 的障碍: 归一化分母需要整行的 exp 和。

2.2 两块合并: alpha 从哪来

把 key 序列切成两块, 先算第 1 块:

```
\nm_1 = max(x[块1]), l_1 = \sum_{j \in \text{块1}} exp(x_j - m_1), o_1 = \sum_{j \in \text{块1}} exp(x_j - m_1) \cdot v_j\n
```

第 2 块到来时, 新的全局最大值 $m_{\text{new}} = \max(m_1, m_2)$ 。关键观察——分母可以重缩放而不是重算:

```
\nl_new = \sum_{j \in \text{块1}} exp(x_j - m_new) + \sum_{j \in \text{块2}} exp(x_j - m_new)\n= \sum_{j \in \text{块1}} exp(x_j - m_1) \cdot exp(m_1 - m_new) + l_2\n= l_1 \cdot \alpha + l_2 其中 \alpha = exp(m_1 - m_new)\n
```

输出的分子部分同理乘 α : $o_{\text{new}} = o_1 \cdot \alpha + o_2$ 。最终 $\text{out} = o_{\text{new}} / l_{\text{new}}$ ——因为 softmax 的分子分母同乘 $\exp(-m_{\text{new}})$, 结果不变 (这就是"减最大值"技巧的推广: 最大值可以事后修正)。

- > online softmax 三状态: 行最大值 m 、分母滚动和 l 、输出累加 acc 。
- > 每来一个新块: 更新 $m \rightarrow$ 算 α 把历史 l/acc 折算到新基准 $\rightarrow$ 累加新块。逐块进行,
- > 整行从不完整出现。数学细节见 FA1 论文 § 3.1 (算法 1)。

2.3 exp2 技巧: 还差一个工程优化

GPU 上 $\text{exp2}(2^x)$ 比 $\text{exp}(e^x)$ 快一个量级 (exp 通常是 exp2 加乘法的宏展开)。利用 $\text{exp}(x) = 2^{(x \cdot \log_2 e)}$, 把换底系数提前折叠进 scale:

```
\npython\nqk_scale = sm_scale * 1.44269504 # scale \times \log_2(e), 只乘一次\nqk_scaled = qk * qk_scale # 此后所有分数都在"log2 域"\np = tl.math.exp2(qk_scaled - m_new) # m_new 也是 log2 域的最大值, 直接相减\nalpha = tl.math.exp2(m_i - m_new) # 同域相减, 无需再乘系数\n
```

这就是内核里那 5 行核心的来历 (下一节逐行讲)。官方 Triton tutorial 06 与 t-vi 的 GPU MODE 讲座都用这个技巧。

三、内核实现 ([scripts/09_flash_attention_triton.py](../scripts/09_flash_attention_triton.py))

3.1 数据流与形状

一个 program 负责 O 的一块 (BLOCK_M 行)，网格 $(\text{cd}iv(T, \text{BLOCK_M}), B \times H)$ ：

```
`  
q 块 (BLOCK_M, D) ←—— 一次载入， 全程驻留寄存器/SMEM  
|  
▼ 对每个 K/V 块 (BLOCK_N 列)：  
kT (D, BLOCK_N) ——tl.dot——► qk (BLOCK_M, BLOCK_N) fp32  
| × qk_scale(已含log2e) + mask(-1.0e6)  
▼  
m_new = max(m, 行最大) ——► α = exp2(m - m_new)  
p = exp2(qk_scaled - m_new) (BLOCK_M, BLOCK_N)  
| .to(bf16)  
v (BLOCK_N, D) ——tl.dot——► acc = acc · α + p @ v (BLOCK_M, D) fp32  
l = l · α + Σ p (BLOCK_M,)  
| 所有块扫完  
▼  
out = acc / l ——► 写回 O 的这一块 (BLOCK_M, D) bf16  
`
```

状态 $m/l/acc$ 全程 fp32；进 `tl.dot` 的 `qk`、`p`、`v` 是 bf16 (Tensor Core 要求低精度输入 + fp32 累加，这正是 04 章"bf16 autocast 几乎不掉点"的硬件根基)。

3.2 内核核心逐行解释

```
`python  
---- online softmax 的 5 行核心 (_fa_fwd_inner 内) ----
```

```
m_new = tl.maximum(m_i, tl.max(qk_scaled, 1)) # ① 行最大值 (fp32)，跨块修正基准  
p = tl.math.exp2(qk_scaled - m_new[:, None]) # ② 稳定 softmax 分子 (未归一，log2 域)  
alpha = tl.math.exp2(m_i - m_new) # ③ 旧累加和的缩放因子 (见 2.2 推导)  
l_i = l_i * alpha + tl.sum(p, 1) # ④ 分母滚动和 (fp32)  
acc = acc * alpha[:, None] + tl.dot(p.to(v.dtype), v) # ⑤ 输出滚动累加  
m_i = m_new
```

- ① `tl.max(qk_scaled, 1)`：沿 key 维 (axis=1) 归约出每行最大值；与历史 `m_i` 取 max 得新基准。首次迭代 `m_i = -inf`， $\alpha = \exp2(-\text{inf} - m_{\text{new}}) = 0$ ，恰好把空历史清零——初始化不需要特判。
- ② 分子只算这一块的，用 `exp2` (换底系数已折叠进 `qk_scale`)。减 `m_new` 保证最大值为 0， $\exp2(0)=1$ ，不溢出。
- ③ 历史 `l/acc` 是按旧基准 `m_i` 算的，换新基准要乘 $\alpha = \exp2(m_i - m_{\text{new}})$ 。注意 $\alpha \leq 1$ (m 只增不减)，且多数块 `m_new == m_i` 时 $\alpha = 1$ 零损耗。
- ⑤ `p.to(v.dtype)`：p 是 fp32，必须降回 bf16 才能和 `v` 一起进 Tensor Core；`acc` 的 fp32 精度由 `tl.dot` 的累加器保证。官方版把 `acc · α` 合并进 `tl.dot(p, v, acc)` 的三参数形式，省一条指令，教学版拆开写更清楚。

3.3 causal 三阶段分解 (对齐官方 tutorial 06)

对第 `start_m` 个 query 行块，K/V 轴分成三段：

key 下标 —►

0 diag_lo 行块末尾 T



▲ query 行块的因果边界 (start_m+1)*BLOCK_M

- 阶段1（带外块）：key 列号全部 < 行块起点，因果条件必然满足——连 mask 判断都省掉，走最快路径（**MASK_MODE=0**，连边界 mask 都不需要：列必在界内）。
- 阶段2（对角块）：块内部分元素越界（列 > 行），需要 **offs_m >= offs_kv** 的逐元素 mask。
- 阶段3：整块全被 mask 掉，根本不进循环——causal 省 ~一半计算量的来源，也是 4K 序列上 causal 比 full 快 ~1.6×（0.279 vs 0.436 ms）的原因。

脚本 09 段 2 打印的实际分解（本次运行 autotune 选中 BLOCK_M=128, BLOCK_N=64；示例块号经钳制取"倒数第二块"，保证任何 BLOCK 组合下都指向真实存在的行）：

[autotune] T=1024 选中 BLOCK_M=128 BLOCK_N=64 num_warps=8 num_stages=3（16 个组合实测选出）
[causal 三阶段] 以第 6 个 query 块（行 768..896）为例：
带外块 [0, 768) 无 mask | 对角块 [768, 896) 逐元素 mask | [896, 1024) 直接跳过

- > △ 对角带起点必须对齐到 BLOCK_N：脚本里 **diag_lo = (start_m*BLOCK_M) // BLOCK_N BLOCK_N**
- > （向下取整）。若直接用 **start_m*BLOCK_M**，当 BLOCK_N > BLOCK_M（如 128 > 64）时，
- > 阶段1的最后一个块会越界扫进对角带，把对角元素算两遍——这正是"误差集中在
- > 特定行"的一类 bug（陷阱 3 详述定位法）。官方 tutorial 的 autotune 网格里
- > **BLOCK_N ≤ BLOCK_M**（截至本文核对的版本），隐式避开了这个问题；我们的网格允许 (64, 128)
- > 组合，必须显式对齐。

3.4 边界处理：other=0 与 -1.0e6

T 不是 BLOCK 的整数倍时（脚本段 3 用 T=1000 专测）：

```
`python
kv_ok = offs_kv < N_CTX # 列越界判断
kT = tl.load(..., mask=kv_ok[None, :], other=0.0) # 越界填 0：进 dot 无害
qk_scaled = qk * qk_scale + tl.where(valid, 0.0, -1.0e6) # 但分数必须挡成"负无穷"
```

两步缺一不可：**other=0** 只保证加载不越界、dot 不出垃圾；但 0 分数经过 **exp2(0 - m)** 会变成正权重混进 l 和 acc，所以还要用 **tl.where** 把非法位置的分数压到 **-1.0e6**，**exp2(-1e6 - m)** 下溢为精确的 0。

- > △ 为什么不用 **-inf**：全非法的行（如 padding 行）会得到 **max = -inf**，
- > 接着 **-inf - (-inf) = NaN**，一次污染整块。**-1.0e6** 足够小（exp2 后精确下溢到 0）
- > 又不会触发 **inf - inf**。这是社区踩了多年的坑（陷阱 1 详述）。

3.5 autotune：02 章那行"Autotuning"落到实处

```
`python
fa_configs = [triton.Config({'BLOCK_M': BM, 'BLOCK_N': BN}, num_warps=w, num_stages=s)
for BM in [64, 128] for BN in [64, 128] for w in [4, 8] for s in [2, 3]]
@triton.autotune(configs=fa_configs, key=['N_CTX', 'HEAD_DIM'])
`
```

16 个组合，首次遇到新 (**N_CTX**, **HEAD_DIM**) 时逐一实测择优。本次运行 T=1024 选中 **BLOCK_M=128 BLOCK_N=64 num_warps=8 num_stages=3**（autotune 的选择随 GPU 状态/时序波动，另一次运行选中过 64/128/4/2——这是正常的）。**num_stages** 是 K/V 加载的软件流水深度（02 章"double buffering"一行的自动版）。

3.6 包装函数

```
`python
def fa_forward(q, k, v, causal=True):
    """q/k/v: (B, H, T, D) bf16 连续 → 输出同形状 bf16（内部 m/l/acc 全 fp32）。"""
    ...
    stage = 3 if causal else 1 # 与官方 tutorial 06 相同的阶段编码
    grid = lambda meta: (triton.cdiv(T, meta['BLOCK_M']), B * H)
    _fa_fwd[grid](q, k, v, o, sm_scale, T, HEAD_DIM=D, STAGE=stage)
`
```

教学版做了两个简化（工业版都支持）：要求连续布局（工业版传 16 个 stride 任意支持）；只做前向且不写 logsumexp（backward 需要它，见官方 tutorial 06 的 **_attn_bwd**）。

四、实测（RTX 4090 / torch 2.6.0+cu124 / triton 3.2.0；2026-09-02，GPU 与其他任务共享）

4.1 SDPA 四后端：锁定并打印实际命中者

不假设、用 profiler 抓 kernel 名作证据（段 1 输出）：

```
`
锁定后端 实际命中（profiler 证据 kernel）

OK flash -> flash | void pytorch_flash::flash_fwd_kernel<...
OK efficient -> efficient | fmha_cutlassF_bf16_aligned_64x64_rf_sm80(...)
OK cudnn -> cudnn | cudnn_generated_fort_native_sdpa_sm80_knob_6_...
OK math -> math | void at::native::elementwise_kernel<128, 2, ...

[默认调度（不锁定）] 命中 flash | void pytorch_flash::flash_fwd_kernel<...
`
```

- bf16 + D=64 + causal 下，4090 上四个后端全部可用，默认调度命中 flash。
- 判别特征：kernel 名里的 **flash** / **fmha** / **cudnn** / (**gemm**+**softmax** = **math** 拆成的多个 eager kernel)。No available kernel 的 RuntimeError 则表示锁不住（比如 fp32 输入锁 flash 就会失败——可以自己试试）。

4.2 数值验收（段 3 输出）

与 naive fp32 参考对照, `rtol=atol=1e-2` (与官方 tutorial 一致) :

```
[T=1024 causal=True] assert_close PASS | max|Δ| 7.70e-03 | 相对误差(|ref|>=0.1 处) 1.6e-02 (SDPA flash 同口径 1.6e-02)
[T=1024 causal=False] assert_close PASS | max|Δ| 9.30e-04 | 相对误差(|ref|>=0.1 处) 6.6e-03 (SDPA flash 同口径 6.2e-03)
[T=1000 causal=True] assert_close PASS | max|Δ| 8.18e-03 | 相对误差(|ref|>=0.1 处) 1.4e-02 (SDPA flash 同口径 1.4e-02)
[T=1000 causal=False] assert_close PASS | max|Δ| 1.06e-03 | 相对误差(|ref|>=0.1 处) 6.2e-03 (SDPA flash 同口径 5.4e-03)
[泄漏检查] 改动 v[j>i]: 行 <=i 输出最大变化 = 0.00e+00 (应=0), 行 >i 最大变化 = 0.57 (应>0) -> 无泄漏 OK
```

- > 误差从哪来: T=1000 (非 64 的倍数) 专测边界 mask, 与 T=1024 同样通过。
- > 相对误差与 SDPA flash 完全同一量级——它由 bf16 输入量化 (尾数仅 8 位, ~0.4%起步) + exp2/scale 折叠顺序主导, 属于预期; "逐元素相对误差"要在 $|ref| \geq 0.1$ 的
- > 区间算才有意义 ($|ref| \rightarrow 0$ 处分母失真, SDPA 也一样大, 那部分靠 atol 兜住)。
- > 泄漏检查: 改动未来位置的 v, 历史行输出逐位不变——causal 语义正确。

4.3 性能 (段 4 输出, 预热 10 + 测 50, `torch.cuda.Event`)

```
T= 1024 causal | naive 0.185 ms | triton 0.031 ms ( 69.3 TF) | best flash 0.031 ms ( 69.0 TF) -> 100.5% 优秀
T= 1024 full  | naive 0.158 ms | triton 0.033 ms (131.9 TF) | best cudnn 0.031 ms (139.4 TF) -> 94.6% 优秀
T= 2048 causal | naive 1.807 ms | triton 0.086 ms (100.4 TF) | best flash 0.109 ms ( 79.0 TF) -> 127.1% 优秀
T= 2048 full  | naive 1.041 ms | triton 0.113 ms (151.7 TF) | best cudnn 0.108 ms (159.5 TF) -> 95.1% 优秀
T= 4096 causal | naive 6.198 ms | triton 0.279 ms (123.3 TF) | best cudnn 0.308 ms (111.6 TF) -> 110.5% 优秀
T= 4096 full  | naive 3.579 ms | triton 0.436 ms (157.6 TF) | best cudnn 0.414 ms (165.9 TF) -> 95.0% 优秀

[总评] 最慢场景 94.6% of SDPA 最优后端 -> 优秀 (>85%)
```

验收承诺 (写进脚本输出): 教学版前向吞吐 $\geq$ SDPA 最优后端的 50% 合格、> 85% 优秀。

依据: PyTorch 官方 FlexAttention 博客实测 Triton 路径达 FA2 前向的 90% (A100) ——这是"通用 Triton 内核"离手工调优 CUTLASS 内核的距离上限, 教学版再让一档到 85%。

- > 教学版为什么能反超 (诚实解读): ① 我们只做前向, 不物化 backward 需要的
- > logsumexp (SDPA 每次前向都要写它); ② autotune 恰好在被测的 (N_CTX, HEAD_DIM)
- > 上选优; ③ B=2/H=8/D=64 的"小"形状下, SDPA 通用内核的固定开销占比大。换 D=128、
- > 大 batch、或加上 backward, FA2 类实现会重新拉开——看量级, 别抠个位数。
- > 测量条件: 本节数字为共享 GPU 环境实测; 同一张空闲卡的另一次独立运行 (六场景 105.4%~163.4%, 含 T=1K causal 163.4%——注意该次在 4090 D 上测得, 与主表的 4090 不同卡, 正好示范"跨卡数字不可直接比") ——空闲时手写内核的比例整体更高。比较内核快慢时, 同卡同负载才有可比性; 这正是陷阱 4 的核心。

一张表看懂趋势: 序列越长, naive 的 $O(T^2)$ 越痛 (6.198 ms vs 0.279 ms, $22.2\times$), 而 Triton 版 4K 时到 123-158 TFLOPS——attention 从"被内存墙拖死的 matmul 链"变回了接近 compute-bound 的内核。这就是 02 章"迁移"学习目标的完整闭环。

五、常见陷阱 (症状 $\rightarrow$ 原因 $\rightarrow$ 解法)

陷阱 1：mask 用 `-inf`，输出 NaN

症状：输出含 NaN，或 `max|Δ|` 爆炸；往往集中在 padding 行 / 首块。

原因：`qk_scaled = tl.where(valid, qk, -inf)` 后，若某行所有位置都被 mask（padding 行、或 bug 导致 mask 写反），`m_new = max(-inf, -inf) = -inf`，随后 `p = exp2(-inf - (-inf)) = exp2(NaN) = NaN`，`α` 同理，一次污染整块。

解法：用足够大的负数 `-1.0e6`（fp32 下 `exp2(-1e6)` 精确下溢到 0），见 3.4 节。真正合法的行永远至少有一个有效位置（自身），不会触发全 mask；padding 行的垃圾结果靠 store 的行 mask 丢弃。

陷阱 2：用 bf16 存 softmax 状态 / 累加器

症状：长序列下误差显著大于 SDPA（ $>1e-1$ 量级），短序列却正常。

原因：`l` 是上千个 ≤ 1 的数相加，`acc` 是 T 次 dot 累加——bf16 只有 8 位尾数，每次累加丢 $\sim 0.4\%$ ，误差随块数线性增长。Tensor Core 的设计用法就是**低精度输入 + fp32 累加**（`tl.dot` 输入 bf16、累加器 fp32 是免费的，反而更快）。

解法：`m_i / l_i / acc` 一律 `tl.zeros(..., dtype=tl.float32)`；只在进 `tl.dot` 前把 `p` 降回 bf16（`p.to(v.dtype)`），写出结果前再转输出 dtype。

陷阱 3：误差集中在特定行 —— causal 块边界 bug 的定位法

症状：整体 `assert_close` 勉强过，但逐行看误差扎堆在某些固定行号。

定位法（先量、再猜，呼应 03 章的 profiling 思维）：

```
`python
err = (tri.float() - ref).abs().amax(dim=-1) # (B, H, T) 每行最大误差
worst = err.flatten(-2).argmax(-1) # 最差行号
print(worst) # 看它是否落在 BLOCK_M 的倍数附近
`
```

- 误差集中在 `BLOCK_M` 的倍数附近 → 块边界 bug：查 `diag_lo` 是否按 `BLOCK_N` 对齐（3.3 节的 `// BLOCK_N * BLOCK_N`）、对角 mask 的比较方向（`>=` 写成 `>`）。
- 误差均匀分布 → 数值路径问题（dtype、累加精度、scale 折叠），走陷阱 2 排查。
- 只在 T 非 `BLOCK` 倍数时出现 → 边界 mask：查 `kv_ok` 是否同时用于 load 和 where。

陷阱 4：benchmark 里的隐形不公平

症状：SDPA 被“锁后端”计时后莫名变慢 10-20%。

原因：把 `with sdpa_kernel([be])` 写进了被计时的 lambda——每次迭代都付一遍上下文进出（backend 开关）的 Python 开销。

解法：上下文管理器包在 `bench()` 外面，只进一次；内核外的开销在微秒级内核对比里是决定性的。同理预热不可省（首次调用含编译/autotune）。

六、生态：FlexAttention 与 SageAttention（段 5）

6.1 FlexAttention：几行 PyTorch 复现同样的 mask

```
`python
from torch.nn.attention.flex_attention import flex_attention, create_block_mask

def causal_mod(b, h, q_idx, kv_idx): # 返回 True = 参与注意力
    return q_idx >= kv_idx

def sliding_mod(b, h, q_idx, kv_idx, W=256): # causal + 最近 W 个 token
    return (q_idx >= kv_idx) & (q_idx - kv_idx <= W)

causal_bm = create_block_mask(causal_mod, None, None, T, T) # 编译成 128x128 块级 Bitmap
sliding_bm = create_block_mask(sliding_mod, None, None, T, T)
out = flex_attention(q, k, v, block_mask=sliding_bm)
```

实测（段 5a 输出）：causal 与 sliding(W=256) 都与 naive 参考 `assert_close` 通过（本次运行最大相对误差 3.96e-02, bf16 同口径）。`create_block_mask` 把 Python 函数变成块级 Bitmap，sliding 这类带状 mask 的全 0 块直接不进内核——正是我们三阶段分解里“阶段 3 跳过”的通用化。FlexAttention 经 `torch.compile` 生成 Triton 内核，官方实测为 FA2 前向的 90%（A100）——它就是“本章内核的自动化版本”。

本机计时补充（RTX 4090 / bf16 / B=2 H=8 D=64 / 共享 GPU / 预热 10 + 测 50, 2026-09-02）：causal 场景 flex_attention 对锁定 flash 后端的 SDPA——T=1024：flex 0.033 ms vs flash 0.031 ms（为 flash 的 108% 耗时）；T=4096：flex 0.434 ms vs flash 0.461 ms（94%，共享 GPU 波动下两者互有胜负）。与官方“90% of FA2”口径一致：flex 的收益在表达力（几行 Python 写任意 mask/score 修改）而不在超越手写内核的速度。

6.2 SageAttention：4090 的甜点（只介绍，不实现）

论文 [arXiv 2411.10958](https://arxiv.org/abs/2411.10958) (SageAttention2)：QK^T 用 INT8、PV 用 INT4 量化并平滑 outlier。4090 恰是甜点卡——RTX 40 系的 INT4 Tensor Core 吞吐极高（论文实测 INT8 只有 INT4 一半速度），整体约 3× FA2。代价是近似（量化误差），推理推荐、训练慎用。与 FA3 的“Hopper 专属 WGMMMA + FP8”是两条不同的路线：一条榨指令集，一条榨数值格式；我们 SM89 的卡只能走第二条。

学完本章你能...

- 推导 online softmax 的 α -重缩放更新（m/l/acc 三状态）
- 手写带 causal 三阶段、边界 mask、exp2 技巧的 Triton Flash Attention 前向
- 用 profiler 证据回答“SDPA 到底命中了哪个后端”
- 说出 FA1→FA3→Flex→Sage 各自的改进点和 4090 的边界（SM89 无 WGMMMA）
- 按验收承诺（≥50% 合格 / >85% 优秀）给自己的内核打分

练习与思考

概念检验

<details>

<summary>Q1: online softmax 为什么必须同时保留 m（行最大值）？只留 l 不行吗？</summary>

A: 不行。l 是"以当前基准 m 为底"的指数和，新块到来若最大值变大 ($m_{\text{new}} > m$)，历史 l 必须乘 $\alpha = \exp(m - m_{\text{new}})$ 折算到新基准，否则历史项被系统性高估。只留 l 就丢失了"历史项是按哪个基准算的"这一信息。反过来说，m 不变时 $\alpha = 1$ 零损耗，所以 m 的作用是让分块流式计算保持数值稳定（防 exp 上溢），这正是它从 Part 1 的"减最大值"一脉相承的地方。

</details>

<details>

<summary>Q2: Flash Attention 省了显存，省了 FLOPs 吗？它到底省了什么？</summary>

A: 不省 FLOPs—— QK^T 与 PV 的乘加数不变（精确算法）。它省的是 HBM 读写：naive 要把 (T,T) 的 S 写回显存、读回来做 softmax、再写回读回来做 PV， $O(T^2)$ 流量；FA 把这一切留在片上（SRAM/寄存器），HBM 流量降到 $O(T \cdot D)$ 。所以 FA 论文标题里有 "IO-Awareness"：在内存墙语境（02 章）下，减少访存比减少计算更值钱。附带收益是训练显存 $O(T^2) \rightarrow O(T)$ ，长序列从"放不下"变"放得下"。

</details>

<details>

<summary>Q3: 实测 causal 只有 full 的 $\sim 1.6 \times$ （0.279 vs 0.436 ms），不是理论上的 $2 \times$ 。差在哪？</summary>

A: 三块不减的开销：① 每个 program 的固定成本——Q 块加载、epilogue 除法与写回都是 $O(\text{BLOCK_M} \cdot D)$ ，与扫多少 key 块无关；② 对角块（阶段 2）仍要全量算再 mask，FLOPs 没省一半；③ 网格/启动开销。带外块（阶段 1）确实省成了"无 mask 快速路径"，加上阶段 3 整段跳过，总账就是 $\sim 1.6 \times$ 。序列越长、BLOCK_M 相对越小，越接近 $2 \times$ 。

</details>

动手实践

练习 1：把内核接回 minimind，替换 SDPA 测 ppl

把 `fa_forward` 接进 [Part7 脚本 05](../Part7_minimind/scripts/05_full_model.py) 的 `MiniMindAttention`，用训练好的 checkpoint 对比验证集 ppl。

步骤提示：

```
`python
```

Part7 里 q/k/v 是 (B, T, H, D)，且 K/V 是 GQA 的 4 个头——先转成内核要的布局：

```
q_h = q.transpose(1, 2) # (B, T, H, D) -> (B, H, T, D)
```

GQA：把 4 个 kv 头 repeat 成 8 个（`torch.repeat_interleave`）再喂 `fa_forward`

或者：8 个 q 头按 kv 分组循环调用（更省显存）

,

验收标准：

- [] 验证 ppl 与 SDPA 版相差 < 0.02 (bf16 噪声级)
- [] 前向耗时与 SDPA flash 同量级（不要求更快）

- [] 写一句结论：GQA 下你的调用方式浪费在哪（提示：repeat 了 K/V）

练习 2：关掉 autotune，画 BLOCK 尺寸的性能曲线

固定 `BLOCK_M/BLOCK_N` 手动传参（把 `@triton.autotune` 去掉或直接调 `_fa_fwd.run`），扫 $(BM, BN) \in \{64, 128\}^2 \times T \in \{1K, 2K, 4K\}$ ，matplotlib 画柱状图（图表标题用英文，如 "FA throughput vs block size"）。

验收标准：

- [] 复现 autotune 的选择在你的机器上确实（接近）最优
- [] 找出最差组合并解释：SMEM 用量（ $BM \cdot BN$ 越大流水线越深）vs occupancy 的权衡
- [] 图上注明环境（GPU / dtype / 独占与否）

扩展思考

- backward 怎么写？前向存下 $m + \log(l)$ （官方 tutorial 06 的 M 矩阵），反向沿同样的分块重算 p 并累计 $dq/dk/dv$ 。试试读懂官方 `_attn_bwd`，画出它与前向对称的"行块/列块"分工图。
- GQA/KV Cache 与本章内核怎么组合？Part 7 的 KV Cache 让 K/V 长度 $\neq$ Q 长度，本章内核的循环边界要怎么改？（提示：Q 块的因果边界从 `offs_m` 变成 `offs_m + kv_offset`。）

参考资料

- [Triton 官方 tutorial 06: fused-attention](https://triton-lang.org/main/getting-started/tutorials/06-fused-attention.html) —— 本章内核的出处（含 backward 与 fp8 变体）
- [GPU MODE Lecture 12: Flash Attention](https://www.youtube.com/watch?v=zEuwuCTEf_0)（主讲 Thomas Viehmann，2024-03-30；[讲义代码](https://github.com/gpu-mode/lectures)在 `lecture_012` 目录）—— 从 online softmax 直觉讲到 Triton 实现，本章的"视频版"
- [tspeterkim/flash-attention-minimal](https://github.com/tspeterkim/flash-attention-minimal) —— 极简单文件实现，适合逐行对照
- [FlashAttention-2 in CuTe, from Scratch (Edwin Chen, echen.io)](https://blog.echen.io/p/flashattention-2-in-cute-from-scratch/) —— 用 CUTLASS CuTe 从零写 FA2 的系列，读懂它就摸到 FA3 的大门
- 论文：FA1 [2205.14135](https://arxiv.org/abs/2205.14135) / FA2 [2307.08691](https://arxiv.org/abs/2307.08691) / FA3 [2407.08608](https://arxiv.org/abs/2407.08608) / online softmax 收敛形式 Rabe & Staats [2112.05682](https://arxiv.org/abs/2112.05682) / SageAttention2 [2411.10958](https://arxiv.org/abs/2411.10958)
- [FlexAttention 官方博客](https://pytorch.org/blog/flexattention/)（90% FA2 的出处）与 [torch.nn.attention.flex_attention 文档](https://docs.pytorch.org/docs/stable/nn.attention.flex_attention.html)

下一步

到这里，Part 9 的闭环完成：**从 vector add 到 matmul 阶梯，从 Triton softmax 到 Flash Attention**——Part 7 里那个"黑盒" `scaled_dot_product_attention`，你已经把它的内部亲手写了一遍。回到课程主线（04 章"路线 C"）：带着内核视角去看 Part 10 的

分布式训练与 Part 14 的 vLLM 推理引擎，你会看到同一个内核出现在不同的系统位置上。

[\[← 上一章：04 Triton 与 PyTorch 扩展\]\(04_triton_and_extensions.md\)](#) | [\[返回 Part 9 目录\]\(README.md\)](#)